INFORME TÉCNICO

Análisis y mejora de seguridad de una aplicación Android

Proyecto: "JaimeMaps"

Estudiante: Jaime Flores, Catalina Barraza, Esteban Gomez , Leander Rodriguez.
Asignatura: Programacion Android.
Docente: Oscar Monardez
Institución: Instituto Tecnico Profesional Santo Tomas.
Fecha de entrega: 30/10/2026

1. Introducción

La seguridad de las aplicaciones móviles es fundamental para proteger la información de las personas usuarias, mantener la integridad de los datos y reducir el riesgo de accesos no

autorizados. Este informe presenta el análisis y la documentación de las medidas de seguridad aplicadas a la aplicación Android JaimeMaps, de acuerdo con el README.md del proyecto.

El README identifica tres áreas de riesgo: inyección SQL, comunicación no segura y exposición de datos sensibles. También declara mejoras relacionadas con el cifrado de información, el uso de HTTPS y la validación y sanitización de entradas. La implementación efectiva de cada medida se comprueba mediante la revisión del código y pruebas documentadas.

Además de la revisión del código, el equipo realizó un análisis estático y uno dinámico del APK de depuración (app-debug.apk) con la herramienta MobSF v4.6.1 el 28/10/2026. Sus resultados se integran en este informe (secciones 4.4 a 4.6, 6 y Anexo A) y complementan, sin reemplazar, el análisis del código fuente.

2. Objetivos

2.1 Objetivo general

Analizar y documentar las vulnerabilidades de seguridad de una aplicación Android y las medidas aplicadas para mitigarlas.

2.2 Objetivos específicos

- Identificar y describir las vulnerabilidades contempladas en el README.md.
- Explicar las medidas técnicas aplicadas para reducir los riesgos.
- Documentar las pruebas realizadas y sus resultados con evidencias verificables.
- Revisar la documentación complementaria y proponer acciones de mejora continua.
- Ejecutar un análisis estático y dinámico con MobSF, interpretar sus hallazgos y distinguir riesgos propios de la aplicación de comportamientos esperados de las librerías de Google.
- Registrar el aporte de cada integrante del equipo y el uso de herramientas de apoyo (prompts).

3. Descripción del proyecto

Nombre del proyecto: JaimeMaps (paquete com.example.jaimemaps, versión 1.0).

Propósito y funcionalidad principal: aplicación Android desarrollada en Java que utiliza Google Maps para mostrar la ubicación del dispositivo y gestionar marcadores. Permite visualizar un mapa, consultar la ubicación actual, guardar marcadores con un nombre definido por la persona usuaria, tomar una fotografía con la aplicación de cámara del sistema y eliminar los marcadores guardados. Está pensada para personas que desean registrar lugares de interés en su propio dispositivo; no existe un servidor ni una API propia.

Tecnologías utilizadas: Android Studio; lenguaje Java (compatibilidad Java 11); Android Gradle Plugin 9.3.3; compileSdk 37, targetSdk 37 y minSdk 30; Google Maps SDK (play-services-maps 19.0.0) y servicios de ubicación (play-services-location 21.3.0); SQLite para el almacenamiento

local de marcadores; AndroidX Biometric 1.1.0; Network Security Configuration; pruebas unitarias con JUnit 4.13.2.

Alcance del análisis: pantalla principal (MainActivity), ingreso y validación del nombre del marcador (InputValidator), acceso a SQLite (MarkerDbHelper), autenticación (AuthHelper con BiometricPrompt), configuración de red (network_security_config.xml), manifiesto (AndroidManifest.xml) y APK de depuración analizado con MobSF. No se revisó infraestructura de servidor porque la aplicación no tiene API propia.

3.1 Equipo de trabajo y roles

El proyecto fue desarrollado por un equipo de cuatro integrantes. La siguiente tabla resume el rol de cada persona y su aporte al desarrollo y a este informe. El detalle de actividades se encuentra en el Anexo C.

Integrante	Rol	Aporte principal	Componentes / pruebas
Jaime Flores	Coordinación técnica y análisis MobSF	Coordinó el proyecto y la integración final del informe. Ejecutó el análisis estático y dinámico con MobSF (Frida Gadget, emulador Pixel 5 API 30) y consolidó los hallazgos.	Análisis MobSF P-07, P-08, P-09 Secciones 4.4 a 4.6
Catalina Barraza	Seguridad de datos y SQL	Implementó la validación de entradas y las consultas parametrizadas. Diseñó y ejecutó las pruebas de inyección y de entradas inválidas.	InputValidator.java MarkerDbHelper.java InputValidatorTest P-01, P-03, P-04
Esteban Gomez	Autenticación y red	Implementó la autenticación con BiometricPrompt y la reautenticación para acciones sensibles. Configuró la política de seguridad de red.	AuthHelper.java network_security_config.xml AndroidManifest.xml P-02, P-05, P-06
Leander Rodriguez	Documentación y control de calidad	Redactó y revisó la documentación de seguridad, verificó la coherencia entre código, reportes y documentos y mantuvo el registro de evidencias.	README.md best_practices.md security_tips.md security_improvement_program.md Anexos A y D

4. Vulnerabilidades identificadas

4.1 Inyección SQL

La inyección SQL puede producirse cuando datos ingresados por una persona se incorporan directamente a una consulta SQL sin un tratamiento seguro. Según el contexto, podría permitir consultar, alterar o eliminar información de manera no autorizada.

Análisis del proyecto:

- **Componente o archivo revisado:** MarkerDbHelper.java (acceso a SQLite), InputValidator.java (validación de entradas) y los métodos promptMarkerTitle() y saveMarker() de MainActivity.java.
- **Código o comportamiento vulnerable observado:** la versión original de la aplicación no tenía base de datos ni ingreso de texto, por lo que no existía una inyección SQL explotable. El riesgo apareció al incorporar persistencia de marcadores con un nombre escrito por la persona usuaria. El patrón vulnerable, que no se utilizó, sería concatenar el texto en la consulta: db.rawQuery("SELECT * FROM markers WHERE title = '" + texto + "'", null).
- **Impacto potencial:** lectura, modificación o eliminación no autorizada de los marcadores (ubicaciones) guardados en el dispositivo.

Medida aplicada: consultas parametrizadas (ContentValues para insertar; selectionArgs con LIKE ? para buscar; API delete para borrar) y validación del título con lista blanca (letras, números, espacios y los símbolos . , - _, máximo 40 caracteres). Las únicas sentencias construidas como texto (CREATE TABLE y DROP TABLE) son constantes sin datos del usuario.

Evidencia y resultado de la prueba: prueba P-01 (Figura 1). La entrada '; DROP TABLE markers;-- fue rechazada con el mensaje "Nombre inválido" y las cuatro pruebas de InputValidatorTest finalizaron con éxito.

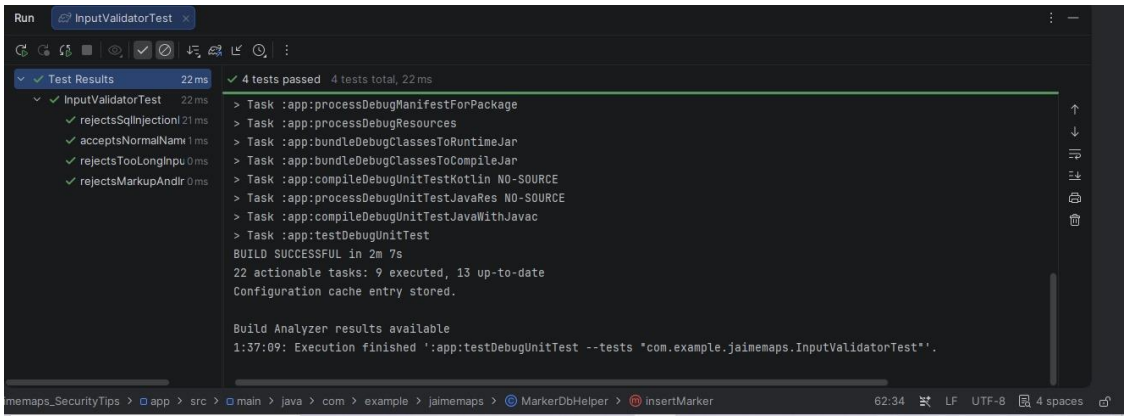


Figura 1a. Resultado de InputValidatorTest en Android Studio — demuestra que las pruebas unitarias de validación aprueban.

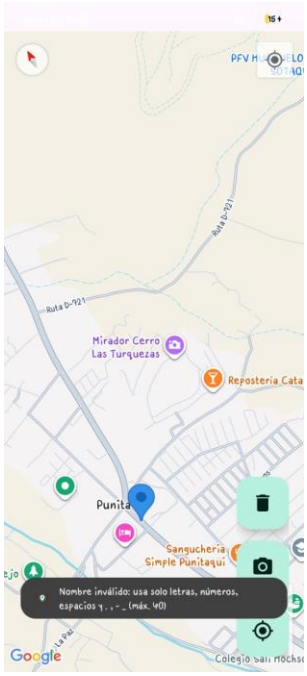


Figura 1b. La aplicación rechaza el nombre malicioso con el mensaje "Nombre inválido" — demuestra que no se guarda el marcador.

4.2 Comunicación no segura

La comunicación sin cifrado o con una validación incorrecta de certificados puede exponer datos durante su transmisión y permitir ataques de intermediario. El uso de HTTPS debe acompañarse de una validación TLS correcta.

Análisis del proyecto:

- **Servicio o URL revisada:** servicios de Google utilizados por la aplicación (Maps SDK y servicios de ubicación). La aplicación no consume ninguna API propia.
- **Configuración inicial detectada:** el proyecto no tenía un archivo network_security_config.xml, por lo que dependía únicamente de los valores por defecto de Android.
- **Corrección implementada:** se agregó res/xml/network_security_config.xml con cleartextTrafficPermitted="false" y confianza solo en certificados del sistema. En AndroidManifest.xml se referenció con android:networkSecurityConfig y se agregó android:usesCleartextTraffic="false". Android moderno ya bloquea HTTP por defecto, por lo que esta medida declara la política de forma explícita y evita que se debilite en el futuro. No se desactivó la validación de certificados ni se definieron TrustManager o HostnameVerifier personalizados.
- **Decisión sobre certificate pinning:** no se fijaron certificados de los dominios de Google porque estos rotan con frecuencia y un pin desactualizado dejaría la aplicación inutilizable. Si en el futuro se incorpora una API propia, se recomienda un pin-set solo para ese dominio.
- **Evidencia y resultado de la prueba:** prueba P-02 (Figura 2). Es una prueba de configuración; el análisis dinámico de MobSF no ejecutó sus pruebas TLS (ver 4.5).

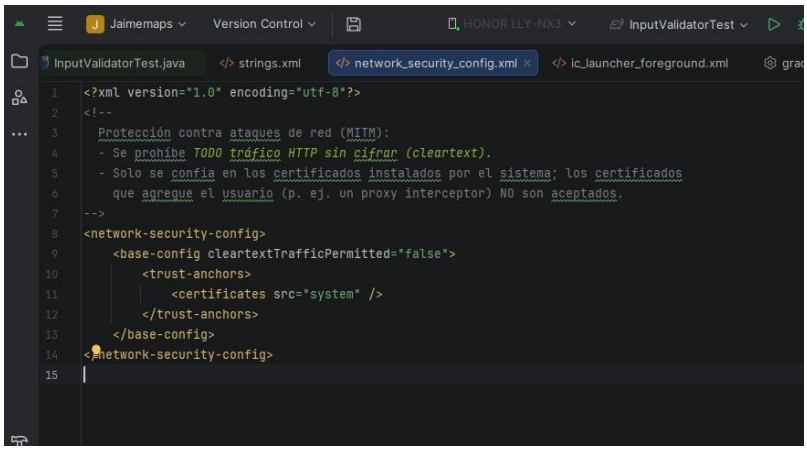


Figura 2a. Archivo network_security_config.xml — demuestra la política sin tráfico en texto claro y con certificados solo del sistema.

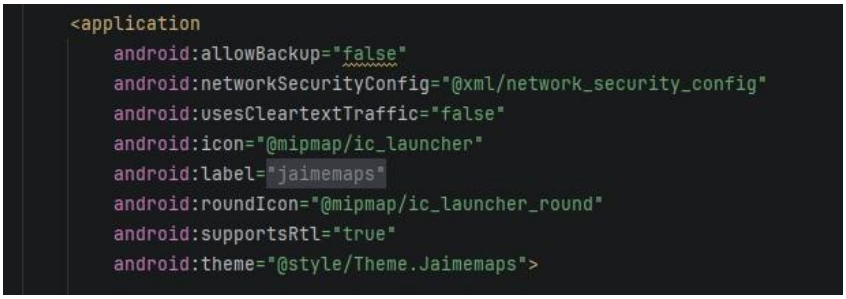


Figura 2b. AndroidManifest.xml — demuestra la referencia a la configuración de red, usesCleartextTraffic="false" y allowBackup="false".

4.3 Exposición de datos sensibles

Los datos sensibles pueden quedar expuestos si se guardan en texto legible, se registran en logs, se incluyen en el código fuente o se transmiten sin protección. Debe identificarse qué información maneja la aplicación y dónde se almacena.

Análisis del proyecto:

- **Tipos de datos revisados:** coordenadas geográficas (latitud y longitud) y nombres de marcadores definidos por la persona usuaria; la clave de API de Google Maps como credencial de la aplicación. La aplicación no maneja contraseñas, PIN ni plantillas biométricas, porque la verificación la realiza el sistema operativo.
- **Ubicación de almacenamiento:** base de datos SQLite local (jaimemaps.db, tabla markers) en el almacenamiento privado de la aplicación. La clave de Google Maps está en res/values/google_maps_api.xml y, por lo tanto, también dentro del APK. No se utilizan Log con ubicaciones ni nombres de marcadores.
- **Medida implementada:** acceso controlado mediante autenticación del dispositivo (la aplicación permanece oculta y no solicita ubicación hasta autenticarse); android:allowBackup="false" en el manifiesto del código actual para evitar la extracción de la base mediante copias de seguridad; ausencia de registros con datos sensibles; permisos de ubicación solicitados en tiempo de ejecución.

- **Evidencia y resultado de la prueba:** prueba P-03 (Figura 3). El análisis dinámico no listó bases de datos propias ni archivos con datos de usuario; solo preferencias del SDK de Maps. Esto no demuestra cifrado: la base de datos local sigue sin cifrar.
- **Pendiente:** revocar o rotar la clave de Google Maps y restringirla por paquete y huella SHA-1 (prioridad alta); cifrar la base SQLite (por ejemplo con SQLCipher, con la clave protegida en Android Keystore) (prioridad media).

Cuando corresponda, se considera Android Keystore para proteger claves criptográficas. No deben almacenarse contraseñas ni claves privadas directamente en el código fuente.

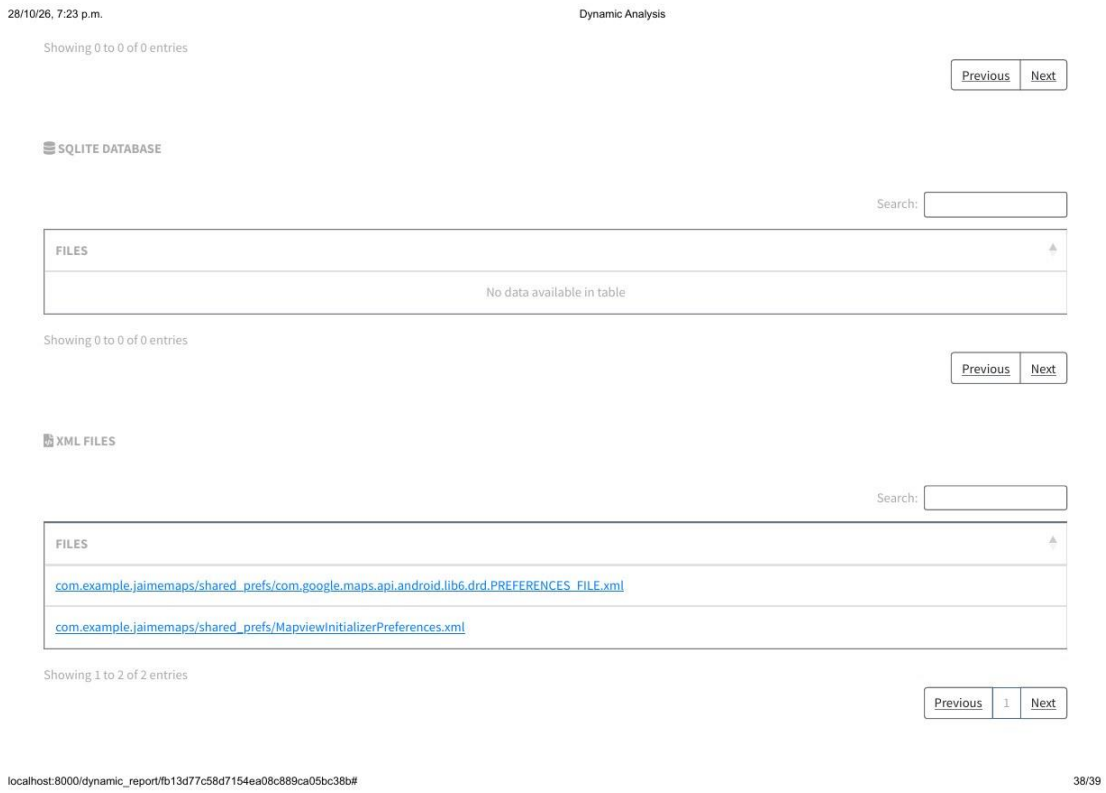


Figura 3. Reporte dinámico de MobSF, secciones SQLite y archivos XML — muestra que solo se listan preferencias del SDK de Maps y no datos de usuario.

4.4 Hallazgos del análisis estático (MobSF)

El análisis estático se ejecutó con MobSF v4.6.1 el 28/10/2026 sobre el archivo app-debug.apk (14,95 MB, target SDK 37, min SDK 30). La puntuación global fue 45/100 (riesgo medio, grado B). La Figura 7 del Anexo A muestra el resumen del reporte. Los hallazgos se detallan en la tabla siguiente, junto con su estado según el código fuente actual del repositorio.

N.º	Hallazgo	Severidad	Evidencia	Impacto y estado actual
1	Clave de Google Maps expuesta en recursos del APK (google_maps_key)	Alta	Sección Hardcoded Secrets del reporte estático. La clave se omite en este informe (AIzaSy…wxopeU) por seguridad.	Permite uso no autorizado de cuotas y costos en Google Cloud. **Pendiente:** sigue definida en google_maps_api.xml; revocar o rotar, restringir por paquete y SHA-1 y sacarla del repositorio.
2	android:debuggable=true	Alta	Manifest Analysis, hallazgo 1 (Figura 8).	Permite conectar depuradores o Frida. Corresponde a la variante debug. **Por verificar:** que la variante release sea debuggable=false.
3	Firma con certificado Android Debug (CN=Android Debug)	Alta	Certificate Analysis (Figura 8).	Un paquete de producción no debe firmarse con certificado de depuración. **Pendiente:** keystore release protegido.
4	android:allowBackup=true	Media / advertencia	Manifest Analysis, hallazgo 2 (Figura 8).	Permite extraer datos con adb backup. **Corregido en el código:** el manifiesto actual declara allowBackup="false" (Figura 2b). Falta repetir el análisis sobre un APK nuevo.
5	Permisos ACCESS_FINE_LOCATION y ACCESS_COARSE_LOCATION	Media / informativa	Application Permissions del reporte estático.	Necesarios para la función de mapa. Se solicitan en tiempo de ejecución, tras la autenticación.
6	Receptor exportado androidx.profileinstaller.ProfileInstallReceiver, protegido por el permiso android.permission.DUMP	Advertencia	Manifest Analysis, hallazgo 3 (Figura 8).	Proviene de una librería de AndroidX, no de código propio. Informativo; revisar en el APK release.

4.5 Hallazgos del análisis dinámico (MobSF y Frida)

El análisis dinámico se ejecutó con Frida Gadget en un emulador Pixel 5 con Android API 30 y las trazas del API Monitor de MobSF. Se interactuó con el mapa y con los controles de la aplicación. Resultados principales:

- **Geolocalización:** el monitor registró llamadas a Location.getLatitude() y getLongitude() desde MainActivity.lambda$getCurrentLocation$0 (línea 106 de MainActivity.java), con las

coordenadas por defecto del emulador. Confirma que la lógica de la aplicación consulta la ubicación para centrar el mapa.

- **Ejecución de comandos:** Runtime.exec ejecutó ["/system/bin/getprop", "debug.mapview.renderer.label"] con código de salida 0, originado en el componente interno com.google.maps.api.android.lib6 de Google Play Services (Figura 9). Es una consulta rutinaria del motor de mapas; no se considera vulnerabilidad explotable.
- **Contenido y archivos:** múltiples consultas de ContentResolver y 74 registros de openFileOutput originados en el SDK de Maps (archivos de caché del mapa dentro del directorio privado de la aplicación).
- **Red:** las conexiones registradas pasan por DelegatingHttpsURLConnection, es decir, HTTPS. Los 10 dominios contactados (por ejemplo mt0.google.com, www.google.com, play.googleapis.com) están clasificados como "good" en la verificación de malware y se ubican en Estados Unidos; no hay dominios en países de la lista OFAC ni rastreadores. Las URL con http:// observadas (generate_204) corresponden a verificaciones de conectividad del sistema y de Google Play Services, no a datos de la aplicación.
- **Pruebas TLS de MobSF:** Cleartext Traffic Test, TLS Misconfiguration Test y las pruebas de TLS Pinning figuran como "Not Tested" (Figura 10). Por ello no se afirma que la validación TLS haya sido probada de forma exhaustiva.
- **Otros módulos:** Exported Activity Tester y Activity Tester sin datos; Clipboard Dump sin información de la aplicación; sin rastreadores.

4.6 Interpretación de los resultados

- **Conteo de hallazgos:** MobSF cuenta 2 hallazgos altos (debuggable y certificado de depuración) y 3 advertencias. El equipo clasifica además la clave de Google Maps como severidad alta por su impacto potencial en costos y cuotas, y registra seis hallazgos en total (3 altos, 2 medios y 1 bajo) en vulnerabilidad.md.
- **APK de depuración:** debuggable=true y la firma Android Debug son normales en una compilación de desarrollo. El riesgo real aparece si esa variante se distribuye; por eso se exige verificar el APK release antes de publicar.
- **Comportamiento de librerías:** getprop, ContentResolver y la escritura de archivos provienen del SDK de Google Maps. Las detecciones de APKiD (por ejemplo, Anti-VM y DexGuard) se registran como información de identificación de librerías y no se interpretan como vulnerabilidades de la aplicación sin una verificación adicional.
- **Desfase entre APK y código:** el APK analizado se compiló antes de algunas correcciones. El manifiesto del código actual ya declara allowBackup="false", pero esto debe comprobarse con un nuevo análisis.

5. Mejoras de seguridad implementadas

Vulnerabilidad	Medida	Archivo/componente	Evidencia	Resultado
Inyección SQL	Consultas parametrizadas y validación con lista blanca	MarkerDbHelper.java, InputValidator.java	Figuras 1a y 1b (P-01)	Entrada maliciosa rechazada; test unitario aprobado
Comunicación no segura	Política HTTPS explícita y confianza solo en certificados del sistema	network_security_config.xml, AndroidManifest.xml	Figuras 2a y 2b (P-02)	Configuración aplicada; la app funciona con la política activa
Datos sensibles	Autenticación para acceder, allowBackup=false, sin Log de datos sensibles	AuthHelper.java, MainActivity.java, AndroidManifest.xml	Figuras 3, 5 y 6 (P-03, P-05, P-06)	Aplicado. La base SQLite sigue sin cifrar (pendiente)
Entradas no válidas	Validación en la aplicación (lista blanca y límite de 40 caracteres)	InputValidator.java	Figura 1b (P-04)	Entradas vacías, con código y de 41 caracteres rechazadas
Acceso no autorizado	BiometricPrompt o credencial del dispositivo, con reautenticación a los 30 s para borrar	AuthHelper.java, MainActivity.java	Figuras 5 y 6 (P-05, P-06)	Autenticación al abrir y antes de eliminar

Los hallazgos del análisis MobSF que siguen abiertos (clave de API, firma release, base sin cifrar, verificación de la variante release) se listan en la sección 9 y en security_improvement_program.md. No se declara como implementada ninguna medida que solo fue investigada o propuesta.

6. Plan de pruebas y evidencias

Las pruebas se realizaron en un entorno propio (emulador y proyecto propio). Se registran el procedimiento, los datos de prueba no sensibles, el resultado esperado, el resultado obtenido y la conclusión.

ID	Prueba	Resultado esperado	Resultado obtenido	Evidencia
P-01	Entradas de prueba para detectar inyección SQL	La entrada no altera la lógica de la consulta y se trata como dato	Al ingresar '; DROP TABLE markers;-- la app mostró "Nombre inválido" y no guardó el marcador. InputValidatorTest pasó sus 4 pruebas.	Figuras 1a y 1b
P-02	Verificar comunicación con el servidor	La conexión utiliza HTTPS y la validación del certificado funciona	La configuración de red está referenciada en el manifiesto con tráfico sin cifrar deshabilitado. La app compila, se ejecuta y el mapa carga con la política activa. No hay servidor propio, por lo que la prueba es de configuración.	Figuras 2a y 2b

ID	Prueba	Resultado esperado	Resultado obtenido	Evidencia
P-03	Revisar almacenamiento de datos sensibles	Los datos y claves no quedan expuestos en texto legible ni en logs	Parcial. En el código no hay Log de ubicaciones ni de nombres y las consultas usan parámetros. El reporte dinámico no listó bases propias ni archivos con datos de usuario. La base SQLite no está cifrada y la clave de Maps está en texto legible en el APK.	Figura 3 y Figura 7
P-04	Validar entradas incorrectas	La aplicación rechaza o gestiona de forma segura los datos no válidos	Un nombre vacío, <script>alert(1)</script> y un texto de 41 caracteres fueron rechazados sin cerrar la app.	Figura 1b
P-05	Ejecutar la aplicación	La aplicación inicia y sus funciones principales operan correctamente	La app inició, pidió autenticación, mostró el mapa, guardó el marcador "Casa" y lo conservó al cerrar y reabrir.	Figura 5
P-06	Verificar autenticación y autorización	El acceso y el borrado de datos exigen autenticación	El sistema pidió autenticación al abrir la app y de nuevo al presionar eliminar pasados 30 s.	Figura 6
P-07	Análisis estático del APK con MobSF	Identificar configuraciones y secretos de riesgo	Puntuación 45/100 (grado B). Hallazgos: clave expuesta, debuggable, firma debug, allowBackup y receptor exportado de AndroidX.	Figuras 7 y 8
P-08	Análisis dinámico con MobSF y Frida	Observar el comportamiento en ejecución sin acciones inesperadas	Se observaron llamadas de geolocalización, el comando getprop del SDK de Maps y conexiones a dominios de Google clasificados como seguros.	Figura 9
P-09	Pruebas TLS del analizador dinámico	Verificar tráfico cifrado y validación de certificados	No concluyente: las pruebas figuran como "Not Tested". Queda como limitación.	Figura 10

Las figuras 1 a 6 se incluyen en las secciones 4.1 a 4.3 y las figuras 7 a 10 en el Anexo A. Ninguna imagen contiene contraseñas, tokens ni claves.

7. Documentación complementaria

7.1 vulnerabilities.md

Registra seis hallazgos (3 altos, 2 medios y 1 bajo) con descripción, severidad, impacto, pasos de reproducción y remediación: clave de API expuesta, android:debuggable=true, firma con certificado de depuración, allowBackup=true, permisos de ubicación y ejecución de comandos del sistema por el SDK de Maps.

7.2 best_practices.md

Reúne las buenas prácticas de validación de entradas, consultas parametrizadas, autenticación y autorización, comunicaciones de red, protección de claves de API, almacenamiento y copias

de seguridad, permisos de ubicación y configuración de compilación y firma. Incluye una tabla de estado: implementadas (validación, SQL parametrizado, autenticación, política de red) y pendientes (clave de Maps, cifrado de SQLite, backup, firma release).

7.3 security_tips.md

Documento ubicado en la raíz del repositorio. Detalla los tres Security Tips implementados: protección contra inyección SQL (consultas parametrizadas y validación de entradas), autenticación y autorización seguras (BiometricPrompt con reautenticación para acciones sensibles) y protección contra ataques de red (política de seguridad de red explícita). Incluye el impacto de cada medida, cómo probarla y sus limitaciones, además de medidas complementarias (allowBackup=false, única Activity exportada, sin Log de datos sensibles).

7.4 security_improvement_program.md

Propone un programa de mejora continua con acciones priorizadas, responsable sugerido, plazo y criterio de verificación: rotar la clave de Google Maps, generar un APK release no depurable con firma de producción, cifrar SQLite, definir la política de backup, revisar la autenticación ante la recreación de la actividad, repetir los análisis MobSF sobre el APK corregido y evaluar R8/ProGuard. Incluye el ciclo identificar, priorizar, corregir, probar, verificar, documentar y revisar.

Los cuatro documentos, junto con el README.md y el reporte consolidado de análisis estático y dinámico, forman la documentación de seguridad del proyecto.

8. Procedimiento de ejecución segura

1. Clonar el repositorio desde la fuente oficial del proyecto.
2. Abrir o importar el proyecto en Android Studio y esperar la sincronización de Gradle.
3. Revisar las instrucciones de compilación, versiones de SDK y dependencias (AGP 9.3.3, compileSdk 37, minSdk 30).
4. Configurar la clave de Google Maps de forma segura para el entorno local, sin incluirla en Git, y restringirla en Google Cloud Console.
5. Ejecutar la aplicación en un emulador o dispositivo de prueba autorizado, con un bloqueo de pantalla (PIN, patrón o contraseña) configurado, porque sin él la aplicación deniega el acceso.
6. Verificar los permisos necesarios y conceder solo los que correspondan a la funcionalidad.
7. Comprobar las funciones principales y realizar las pruebas de seguridad documentadas.

Se registran las versiones del entorno y cualquier error encontrado. No se comparten archivos de configuración que contengan credenciales, tokens o claves.

9. Resultados

- **Resumen:** las pruebas P-01, P-02, P-04, P-05 y P-06 resultaron satisfactorias. P-07 y P-08 se ejecutaron y permitieron identificar y clasificar los riesgos de configuración del APK. P-03 resultó parcial y P-09 no fue concluyente.

- **Limitaciones:** la aplicación no tiene API propia, por lo que las protecciones de red son preventivas; las pruebas TLS de MobSF no se ejecutaron; la base de datos local no está cifrada; la autenticación depende del bloqueo de pantalla del dispositivo; el análisis se hizo sobre un APK de depuración compilado antes de algunas correcciones.
- **Riesgos pendientes (prioridad alta):** revocar o rotar la clave de Google Maps y restringir la nueva por paquete y huella SHA-1; generar y verificar un APK release no depurable firmado con un keystore de producción.
- **Riesgos pendientes (prioridad media):** cifrar la base de datos SQLite; confirmar allowBackup y las reglas de extracción en el APK final; revisar el flujo de autenticación ante la recreación de la actividad; repetir el análisis estático y dinámico sobre el APK corregido.
- **Prioridad baja:** evaluar la reducción y ofuscación con R8/ProGuard (en el build actual la optimización de la variante release está deshabilitada).

10. Conclusiones

Se implementaron tres Security Tips: protección contra inyección SQL, autenticación y autorización mediante credenciales del dispositivo, y política de red explícita. Las pruebas P-01 a P-06 respaldan que las entradas maliciosas son rechazadas, que el acceso y el borrado de datos exigen autenticación, y que la configuración de red está aplicada.

El análisis con MobSF (45/100, riesgo medio) aportó una visión del APK distinta a la del código: reveló la clave de Google Maps expuesta, la compilación depurable, la firma de depuración y el respaldo habilitado. De ellos, el respaldo ya fue corregido en el código; los demás requieren acciones antes de cualquier publicación. Los comportamientos dinámicos observados (getprop, archivos de caché, consultas de contenido) pertenecen al SDK de Google Maps y no se consideran vulnerabilidades explotables por sí solos.

Quedan pendientes el cifrado de la base de datos local, la protección de la clave de API y la verificación de una variante release. Esto no implica que la aplicación sea completamente segura.

Conclusión del proyecto: el trabajo en equipo permitió combinar desarrollo seguro, pruebas y análisis automatizado. La principal lección es que las medidas deben verificarse sobre el APK que se distribuye, no solo sobre el código, y que cada corrección debe respaldarse con evidencia.

11. Referencias

- README.md del proyecto y documentación incluida en el repositorio (security_tips.md, best_practices.md, vulnerabilidad.md, security_improvement_program.md).
- Android Developers, Network security configuration: https://developer.android.com/privacy-and-security/security-config
- Android Developers, Biometric authentication: https://developer.android.com/identity/sign-in/biometric-auth
- Android Developers, Privacidad y seguridad: https://developer.android.com/privacy-and-security

- OWASP Mobile Application Security: https://mas.owasp.org/
- Mobile Security Framework (MobSF): https://github.com/MobSF/Mobile-Security-Framework-MobSF
- Documentación de Google Maps SDK for Android, restricciones de claves de API, y de AndroidX Biometric.

Anexo A. Registro de evidencias

Las figuras 1 a 6 se encuentran en las secciones 4.1 a 4.3. A continuación se presentan las evidencias de los análisis MobSF.

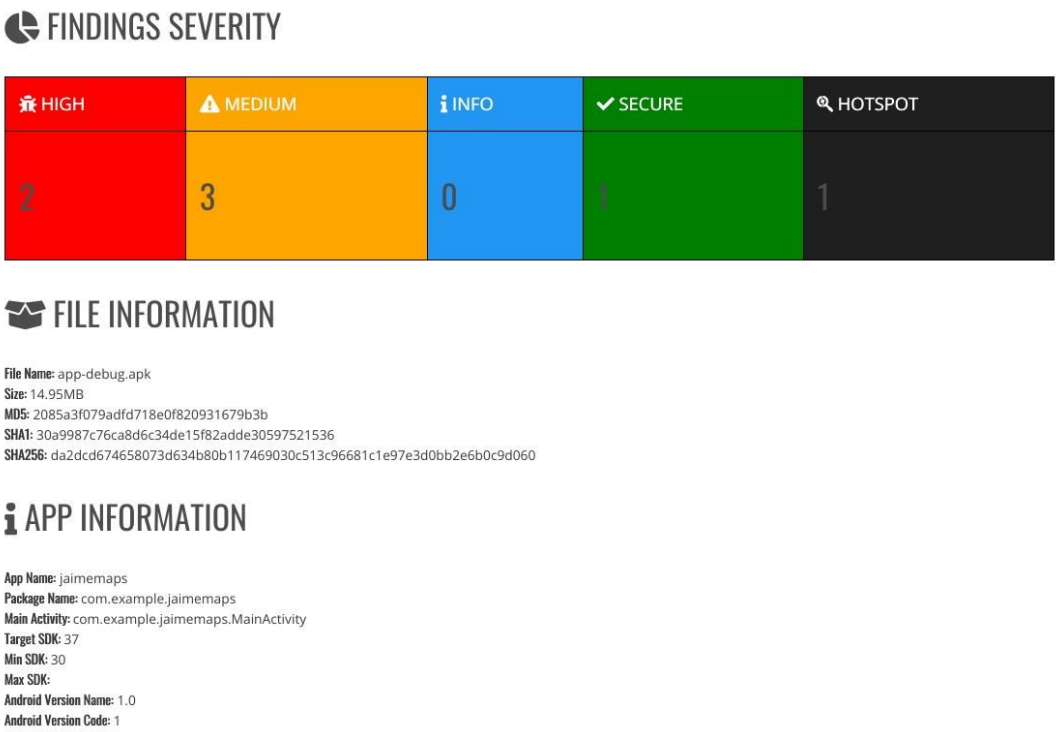


Figura 7. Reporte estático MobSF (resumen) — demuestra la cantidad de hallazgos por severidad y los datos del archivo app-debug.apk analizado el 28/10/2026.

TITLE	SEVERITY	DESCRIPTION
Signed Application	info	Application is signed with a code signing certificate
Application signed with debug certificate	high	Application signed with a debug certificate. Production application must not be shipped with a debug certificate.

MANIFEST ANALYSIS

HIGH: 1 | WARNING: 2 | INFO: 0 | SUPPRESSED: 0

NO	ISSUE	SEVERITY	DESCRIPTION
1	Debug Enabled For App [android:debuggable=true]	high	Debugging was enabled on the app which makes it easier for reverse engineers to hook a debugger to it. This allows dumping a stack trace and accessing debugging helper classes.
2	Application Data can be Backed up [android:allowBackup=true]	warning	This flag allows anyone to backup your application data via adb. It allows users who have enabled USB debugging to copy application data off of the device.
3	Broadcast Receiver (androidx.profileinstaller.ProfileInstallReceiver) is Protected by a permission, but the protection level of the permission should be checked. Permission: android.permission.DUMP [android:exported=true]	warning	A Broadcast Receiver is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device. It is protected by a permission which is not defined in the analysed application. As a result, the protection level of the permission should be checked where it is defined. If it is set to normal or dangerous, a malicious application can request and obtain the permission and interact with the component. If it is set to signature, only applications signed with the same certificate can obtain the permission.

CODE ANALYSIS

Figura 8. Reporte estático MobSF (certificado y manifiesto) — demuestra la firma con certificado de depuración, debuggable, allowBackup y el receptor exportado.



Figura 9. Reporte dinámico MobSF (Command) — demuestra la ejecución de getprop desde el SDK de Google Maps con código de salida 0.



Figura 10. Reporte dinámico MobSF (TLS/SSL Security Tester) — demuestra que las pruebas TLS figuran como "Not Tested".

Evidencia opcional: captura del reporte estático, sección Hardcoded Secrets, con la clave ocultada (por ejemplo, mostrando solo "AIzaSy…wxopeU"), para respaldar el hallazgo 1. No publicar la clave completa.

Anexo B. Datos de ejecución

- Sistema operativo y versión: [Completar].
- Versión de Android Studio: [Completar].
- Lenguaje y versión: Java (compatibilidad 11).
- Android Gradle Plugin: 9.3.3. compileSdk 37, targetSdk 37, minSdk 30.
- Android SDK / dispositivo o emulador: emulador Pixel 5 con Android API 30 para el análisis dinámico; [Completar] para las pruebas funcionales.
- Herramientas de análisis: MobSF v4.6.1 (estático y dinámico) y Frida Gadget.
- Fecha de ejecución de pruebas: 28/10/2026 (análisis MobSF); [Completar] para las pruebas P-01 a P-06.

Anexo C. Aporte de cada integrante

Integrante	Actividades realizadas	Resultado entregado
Jaime Flores	Compilación del APK de depuración y ejecución del análisis estático y dinámico en MobSF. Interpretación de hallazgos y comparación con el código. Integración del informe final.	Reportes MobSF (estático y dinámico) vulnerabilidad.md Secciones 4.4 a 4.6 y Anexo A
Catalina Barraza	Implementación de InputValidator y de consultas parametrizadas en MarkerDbHelper. Pruebas unitarias y manuales de entradas maliciosas e inválidas.	InputValidatorTest Pruebas P-01 y P-04 Sección 4.1
Esteban Gomez	Implementación de AuthHelper (BiometricPrompt) y reautenticación a los 30 s. Configuración de network_security_config.xml y del manifiesto.	Pruebas P-02, P-05 y P-06 Secciones 4.2 y 4.3
Leander Rodriguez	Redacción y revisión de README, best_practices, security_tips y security_improvement_program. Control de coherencia entre documentos y evidencias.	Documentación de seguridad Sección 7 y Anexos A y D

Todos los integrantes revisaron el informe final antes de su entrega.

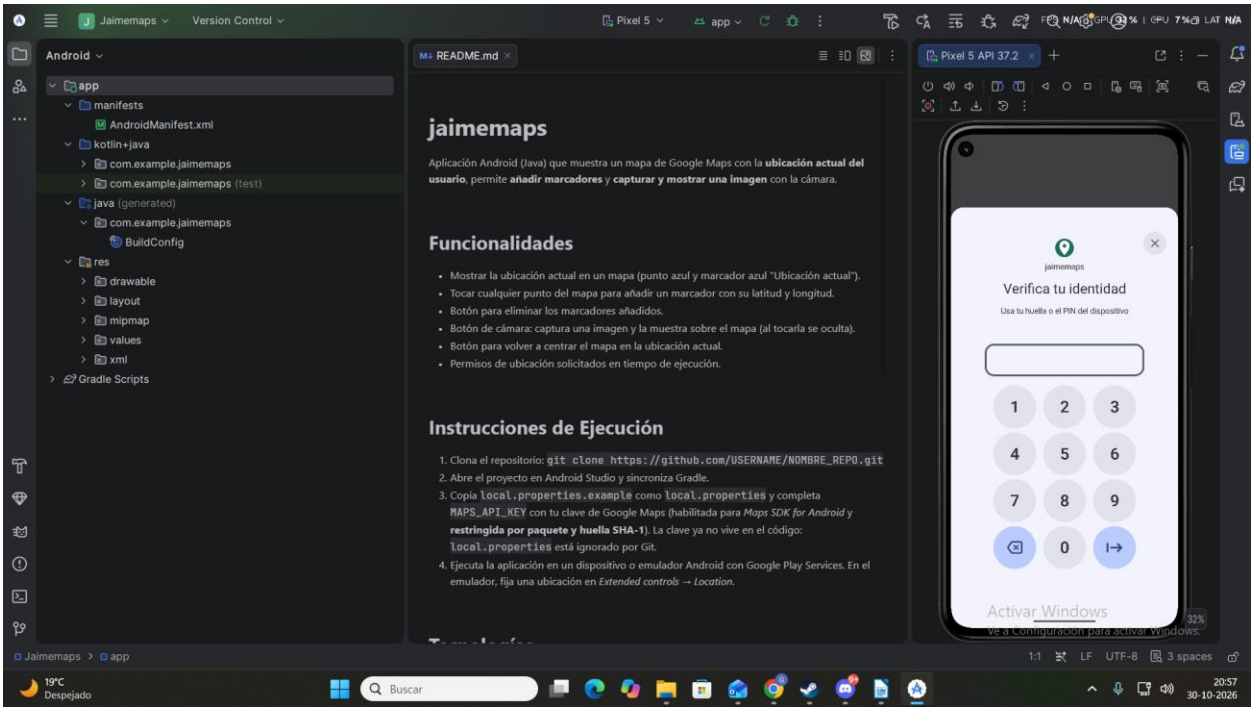
Anexo D. Prompts utilizados

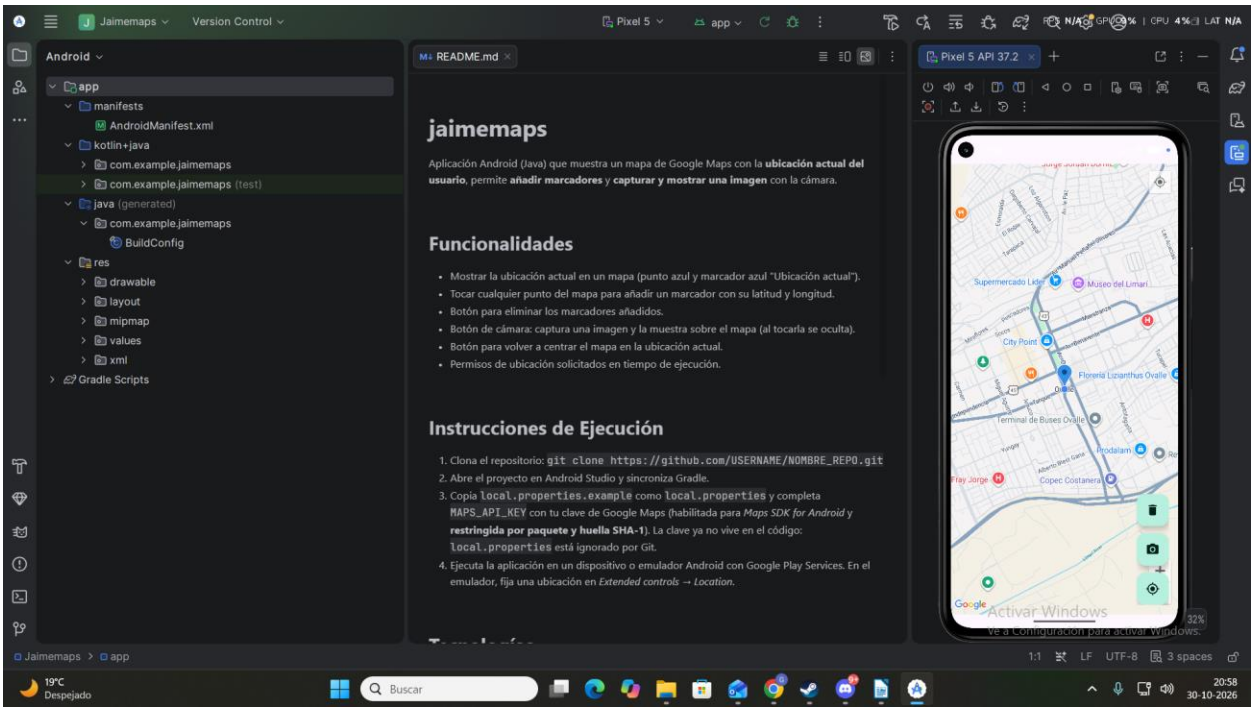
Durante el trabajo se utilizó un asistente de inteligencia artificial (Claude) como apoyo para entender los conceptos, ordenar la información y redactar. Las decisiones técnicas, la ejecución de las pruebas y la verificación de los resultados fueron realizadas por el equipo. A continuación se registran, en lenguaje cotidiano, los prompts de referencia (reconstruidos a partir del proceso de trabajo).

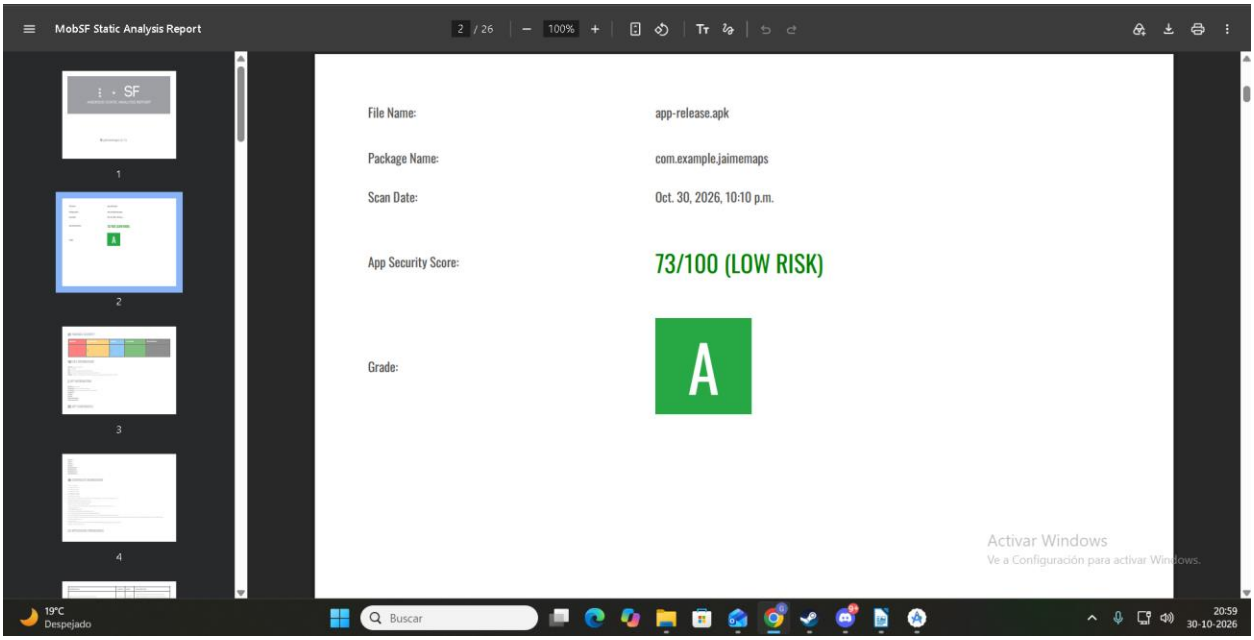
N.º	Etapa	Prompt
1	Inicio del trabajo	"Hola, tengo que hacer un informe de seguridad de mi app de mapas de Android para el ramo. ¿Me explicas qué tengo que hacer y por dónde partir?"
2	Entradas de usuario	"Mi app guarda marcadores con un nombre que escribe la persona. ¿Eso puede ser peligroso? ¿Cómo hago para que no puedan meter cosas raras en el nombre?"
3	Autenticación	"¿Cómo hago que la app pida la huella o el PIN del celular antes de abrirse y otra vez cuando alguien quiera borrar los marcadores?"
4	Red	"Me piden que la app no se conecte de forma insegura a internet. ¿Qué tengo que cambiar para que eso quede bien?"
5	Herramienta de análisis	"Usé una herramienta que revisa apps y me salieron cosas en rojo y amarillo. ¿Me ayudas a entender qué significa cada una y cuáles son graves de verdad?"

N.º	Etapa	Prompt
6	Clave de Google Maps	"Me dice que mi clave de Google Maps quedó a la vista dentro de la app. ¿Qué tan grave es y cómo la protejo?"
7	Compilación de pruebas	"¿Qué quiere decir que la app esté firmada con certificado de depuración y que sea depurable? ¿Eso es malo si es solo para pruebas?"
8	Prueba en vivo	"En la prueba en vivo salió que la app ejecuta un comando llamado getprop. ¿Hice algo mal o es normal?"
9	Orden del informe	"Tengo el informe con espacios vacíos y mis resultados en otros archivos. ¿Me ayudas a juntarlo todo en un solo informe sin perder lo que ya escribí?"
10	Trabajo en equipo	"Somos cuatro en el grupo. ¿Cómo pongo lo que hizo cada uno de forma ordenada?"
11	Pendientes	"Necesito explicar lo que todavía no arreglamos sin decir que la app es 100% segura."
12	Evidencias	"¿En qué parte del informe va cada captura de pantalla de los análisis?"

Referencias:




Jaimemaps
Version Control
Pixel 5
app
Android
app
manifests
AndroidManifest.xml
kotlin+java
com.example.jaimemaps
com.example.jaimemaps (test)
java (generated)
com.example.jaimemaps
BuildConfig
res
drawable
layout
mipmap
values
xml
Gradle Scripts
README.md
jaimemaps
Aplicación Android (Java) que muestra un mapa de Google Maps con la ubicación actual del usuario, permite añadir marcadores y capturar y mostrar una imagen con la cámara.
Funcionalidades
Mostrar la ubicación actual en un mapa (punto azul y marcador azul "Ubicación actual").
Tocar cualquier punto del mapa para añadir un marcador con su latitud y longitud.
Botón para eliminar los marcadores añadidos.
Botón de cámara: captura una imagen y la muestra sobre el mapa (al tocarla se oculta).
Botón para volver a centrar el mapa en la ubicación actual.
Permisos de ubicación solicitados en tiempo de ejecución.
Instrucciones de Ejecución
1. Clona el repositorio: git clone https://github.com/USERNAME/NOMBRE_REPO.git
2. Abre el proyecto en Android Studio y sincroniza Gradle.
3. Copia local.properties.example como local.properties y completa MAPS_API_KEY con tu clave de Google Maps (habilitada para Maps SDK for Android y restringida por paquete y huella SHA-1). La clave ya no vive en el código: local.properties está ignorado por Git.
4. Ejecuta la aplicación en un dispositivo o emulador Android con Google Play Services. En el emulador, fija una ubicación en Extended controls → Location.
Pixel 5 API 37.2
Activar Windows
Ve a Configuración para activar Windows.
Jaimemaps > app
1:1
LF
UTF-8
3 spaces
19°C
Despejado
Buscar
20:58
30-10-2026


MobSF Static Analysis Report
2 / 26
100%
File Name:
app-release.apk
Package Name:
com.example.jaimemaps
Scan Date:
Oct. 30, 2026, 10:10 p.m.
App Security Score:
73/100 (LOW RISK)
Grade:
A
Activar Windows
Ve a Configuración para activar Windows.
19°C
Despejado
Buscar
20:59
30-10-2026